Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.4.1 sur 153 | difficulté 1 / 10

0.4.1. L’alphabet à la main, jusqu’au bout de la ligne — de base, ailleurs

Les vingt lettres du 0.4, sur la ligne 5 au lieu de la ligne 0.

1. Ce qu’il faut comprendre

C’est le 0.4, avec un seul changement : la **ligne** : 5 au lieu de 0, pour les vingt lignes de code.

Seul le deuxième nombre change, dans chaque `texte` : c’est la ligne. Les colonnes restent 0 à 19 : la même ligne de lettres, plus bas.

C’est la version de base : une seule chose, à la deuxième place. Le 0.4.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <texte>` → 0.0.2. La fonction texte() — écrire un mot

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <texte>  // écrit un texte à l'écran`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un texte à l'écran ».)
`int main() {  // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`// Le changement : la ligne, 0 → 5, dans chaque texte.`	Un commentaire : une phrase pour celui qui lit (« Le changement : la ligne, 0 → 5, dans chaque texte. »). La console l'ignore.

`// texte(colonne, ligne, "LETTRE") : le 2e nombre (la ligne) vaut 5 partout ;`	Un commentaire : une phrase pour celui qui lit (« texte(colonne, ligne, "LETTRE") : le 2e nombre (la ligne) vaut 5 partout ; »). La console l'ignore.
`// la colonne va toujours de 0 à 19, une lettre par case.`	Un commentaire : une phrase pour celui qui lit (« la colonne va toujours de 0 à 19, une lettre par case. »). La console l'ignore.
`texte(0, 5, "A");  // colonne 0, ligne 5`	Appel de la fonction texte de la console, avec les arguments 0, 5, "A". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « colonne 0, ligne 5 ».)
`texte(1, 5, "B");  // colonne 1, ligne 5`	Appel de la fonction texte de la console, avec les arguments 1, 5, "B". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « colonne 1, ligne 5 ».)
`texte(2, 5, "C");`	Appel de la fonction texte de la console, avec les arguments 2, 5, "C". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(3, 5, "D");`	Appel de la fonction texte de la console, avec les arguments 3, 5, "D". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(4, 5, "E");`	Appel de la fonction texte de la console, avec les arguments 4, 5, "E". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(5, 5, "F");`	Appel de la fonction texte de la console, avec les arguments 5, 5, "F". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(6, 5, "G");`	Appel de la fonction texte de la console, avec les arguments 6, 5, "G". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec

	une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(7, 5, "H");`	Appel de la fonction texte de la console, avec les arguments 7, 5, "H". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(8, 5, "I");`	Appel de la fonction texte de la console, avec les arguments 8, 5, "I". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(9, 5, "J");`	Appel de la fonction texte de la console, avec les arguments 9, 5, "J". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(10, 5, "K");`	Appel de la fonction texte de la console, avec les arguments 10, 5, "K". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(11, 5, "L");`	Appel de la fonction texte de la console, avec les arguments 11, 5, "L". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(12, 5, "M");`	Appel de la fonction texte de la console, avec les arguments 12, 5, "M". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(13, 5, "N");`	Appel de la fonction texte de la console, avec les arguments 13, 5, "N". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(14, 5, "O");`	Appel de la fonction texte de la console, avec les arguments 14, 5, "O". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18

	lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(15, 5, "P");`	Appel de la fonction texte de la console, avec les arguments 15, 5, "P". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(16, 5, "Q");`	Appel de la fonction texte de la console, avec les arguments 16, 5, "Q". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(17, 5, "R");`	Appel de la fonction texte de la console, avec les arguments 17, 5, "R". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(18, 5, "S");`	Appel de la fonction texte de la console, avec les arguments 18, 5, "S". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`texte(19, 5, "T");  // colonne 19 : la dernière de la ligne`	Appel de la fonction texte de la console, avec les arguments 19, 5, "T". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « colonne 19 : la dernière de la ligne ».)
`while (true) {  // La boucle de jeu : recommence pour toujours...`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « La boucle de jeu : recommence pour toujours... ».)
`image();  // ... attendre l'image suivante.`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « ... attendre l'image suivante. ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **735 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()          227 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          320 o  ← CopierTuiles  — 20 lettres sur 43
  "A"               1 o  ← texte()  — un texte gravé
  "B"               1 o  ← texte()  — un texte gravé
  "C"               1 o  ← texte()  — un texte gravé
  "D"               1 o  ← texte()  — un texte gravé
  "E"               1 o  ← texte()  — un texte gravé
  "F"               1 o  ← texte()  — un texte gravé
  "G"               1 o  ← texte()  — un texte gravé
  "H"               1 o  ← texte()  — un texte gravé
  "I"               1 o  ← texte()  — un texte gravé
  "J"               1 o  ← texte()  — un texte gravé
  "K"               1 o  ← texte()  — un texte gravé
  "L"               1 o  ← texte()  — un texte gravé
  "M"               1 o  ← texte()  — un texte gravé
  "N"               1 o  ← texte()  — un texte gravé
  "O"               1 o  ← texte()  — un texte gravé
  "P"               1 o  ← texte()  — un texte gravé
  "Q"               1 o  ← texte()  — un texte gravé
  "R"               1 o  ← texte()  — un texte gravé
  "S"               1 o  ← texte()  — un texte gravé
  "T"               1 o  ← texte()  — un texte gravé
Lettres gravées : ABCDEFGHIJKLMNOPQRST
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 20   220 o sur place, dans main()  + EcrireTexte (10 o)
  image() ×  1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>   // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

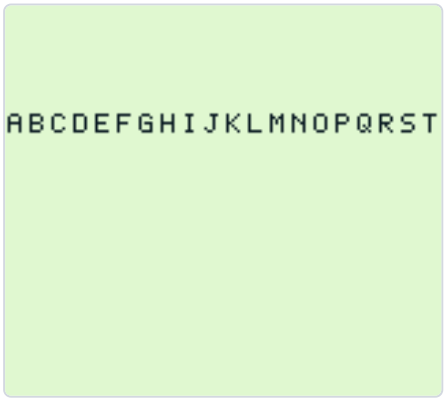
```
int main() {            // Le programme commence ici.
  // Le changement : la ligne, 0 → 5, dans chaque texte.
  // texte(colonne, ligne, "LETTRE") : le 2e nombre (la ligne) vaut 5 partout ;
  // la colonne va toujours de 0 à 19, une lettre par case.
  texte(0, 5, "A");     // colonne 0, ligne 5
  texte(1, 5, "B");     // colonne 1, ligne 5
  texte(2, 5, "C");
  texte(3, 5, "D");
  texte(4, 5, "E");
  texte(5, 5, "F");
  texte(6, 5, "G");
  texte(7, 5, "H");
  texte(8, 5, "I");
  texte(9, 5, "J");
  texte(10, 5, "K");
  texte(11, 5, "L");
  texte(12, 5, "M");
  texte(13, 5, "N");
  texte(14, 5, "O");
  texte(15, 5, "P");
  texte(16, 5, "Q");
  texte(17, 5, "R");
  texte(18, 5, "S");
  texte(19, 5, "T");    // colonne 19 : la dernière de la ligne

  while (true) {        // La boucle de jeu : recommence pour toujours...
    image();            // ... attendre l'image suivante.
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> A à T sur la ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 5 : A à T

Ensuite : **0.4.2. L'alphabet à la main, jusqu'au bout de la ligne — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++